Collab Notebook by Vikram Nadathur

June 14, 2026

1 Main Data Structures in python

1.1 Lists https://www.w3schools.com/python/python_lists.asp

1.2 Exercises and Examples

Create a list of squares from 1 to 10.

[4]:
```
#With List Comprehension
squares = [x**4 for x in range(1, 15)]
print(squares)
# Output: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

#Without List Comprehension:
squares = []
for x in range(1, 11):
    squares.append(x**2)
print(squares)
```

```
# Output: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

```
[1, 16, 81, 256, 625, 1296, 2401, 4096, 6561, 10000, 14641, 20736, 28561, 38416]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

2 Create a list of even numbers from 1 to 20.

[5]:
```
#With List Comprehension
evens = [x for x in range(1, 21) if x % 2 == 0]
print(evens)
# Output: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

#Without List Comprehension:
evens = []
for x in range(1, 21):
    if x % 2 == 0:
        evens.append(x)
print(evens)
# Output: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

2.1 Create a list of names that start with the letter 'A'.

[6]:
```
#With List Comprehension
names = ["Alice", "Bob", "Charlie", "David", "Alex", "Yulia"]
a_names = [name for name in names if name.startswith('Y')]
print(a_names)
# Output: ['Alice', 'Alex']

#Without List Comprehension
names = ["Alice", "Bob", "Charlie", "David", "Alex"]
a_names = []
for name in names:
    if name.startswith('A'):
        a_names.append(name)
print(a_names)
# Output: ['Alice', 'Alex']
```

```
['Yulia']
['Alice', 'Alex']
```

[7]:
```
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

```python
import pandas as pd

# 1D List
one_d_list = [1, 2, 3, 4, 5]


# 2D List
two_d_list = [
    [1, 2, 3, 4, 5],
    [6, 7, 8, 9, 10],
    [11, 12, 13, 14, 15],
    [16, 17, 18, 19, 20],
    [21, 22, 23, 24, 25]
]


# 3D List
three_d_list = [
    [
        [1, 2, 3, 4, 5],
        [6, 7, 8, 9, 10],
        [11, 12, 13, 14, 15],
        [16, 17, 18, 19, 20],
        [21, 22, 23, 24, 25]
    ],
    [
        [26, 27, 28, 29, 30],
        [31, 32, 33, 34, 35],
        [36, 37, 38, 39, 40],
        [41, 42, 43, 44, 45],
        [46, 47, 48, 49, 50]
    ]
]


# 4D List
four_d_list = [
    [
        [
            [1, 2, 3, 4, 5],
            [6, 7, 8, 9, 10],
            [11, 12, 13, 14, 15],
            [16, 17, 18, 19, 20],
            [21, 22, 23, 24, 25]
        ],
        [
            [26, 27, 28, 29, 30],
```

```
            [31, 32, 33, 34, 35],
            [36, 37, 38, 39, 40],
            [41, 42, 43, 44, 45],
            [46, 47, 48, 49, 50]
        ]
    ],
    [
        [
            [51, 52, 53, 54, 55],
            [56, 57, 58, 59, 60],
            [61, 62, 63, 64, 65],
            [66, 67, 68, 69, 70],
            [71, 72, 73, 74, 75]
        ],
        [
            [76, 77, 78, 79, 80],
            [81, 82, 83, 84, 85],
            [86, 87, 88, 89, 90],
            [91, 92, 93, 94, 95],
            [96, 97, 98, 99, 100]
        ]
    ]
]
print(one_d_list)
# 1D List Visualization
fig_1d = px.line(x=list(range(len(one_d_list))), y=one_d_list, labels={'x':␣
 ↪'Index', 'y': 'Value'}, title="1D List Visualization")
fig_1d.show()

print(two_d_list)
# 2D List Visualization
df_2d = pd.DataFrame(two_d_list)
fig_2d = go.Figure(data=go.Heatmap(z=df_2d.values, x=[f'Column {i+1}' for i in␣
 ↪range(df_2d.shape[1])], y=[f'Row {i+1}' for i in range(df_2d.shape[0])]))
fig_2d.update_layout(title='2D List Visualization', xaxis_title='Columns',␣
 ↪yaxis_title='Rows')
fig_2d.show()

print(three_d_list)
# 3D List Visualization
fig_3d = make_subplots(rows=1, cols=len(three_d_list), subplot_titles=[f"Matrix␣
 ↪{i+1}" for i in range(len(three_d_list))])

for i, matrix in enumerate(three_d_list):
    fig_3d.add_trace(
        go.Heatmap(z=matrix, x=[f'Column {j+1}' for j in␣
 ↪range(len(matrix[0]))], y=[f'Row {j+1}' for j in range(len(matrix))]),
```

```
        row=1, col=i+1
    )

fig_3d.update_layout(title_text="3D List Visualization")
fig_3d.show()

print(four_d_list)
# 4D List Visualization
fig_4d = make_subplots(rows=len(four_d_list), cols=len(four_d_list[0]),␣
 ↪subplot_titles=[f"Tensor {i+1}, Matrix {j+1}" for i in␣
 ↪range(len(four_d_list)) for j in range(len(four_d_list[0]))])

for i, tensor in enumerate(four_d_list):
    for j, matrix in enumerate(tensor):
        fig_4d.add_trace(
            go.Heatmap(z=matrix, x=[f'Column {k+1}' for k in␣
 ↪range(len(matrix[0]))], y=[f'Row {k+1}' for k in range(len(matrix))]),
            row=i+1, col=j+1
        )

fig_4d.update_layout(title_text="4D List Visualization")
fig_4d.show()
```

```
[1, 2, 3, 4, 5]
```

```
[[1, 2, 3, 4, 5], [6, 7, 8, 9, 10], [11, 12, 13, 14, 15], [16, 17, 18, 19, 20],
[21, 22, 23, 24, 25]]
```

```
[[[1, 2, 3, 4, 5], [6, 7, 8, 9, 10], [11, 12, 13, 14, 15], [16, 17, 18, 19, 20],
[21, 22, 23, 24, 25]], [[26, 27, 28, 29, 30], [31, 32, 33, 34, 35], [36, 37, 38,
39, 40], [41, 42, 43, 44, 45], [46, 47, 48, 49, 50]]]
```

```
[[[[1, 2, 3, 4, 5], [6, 7, 8, 9, 10], [11, 12, 13, 14, 15], [16, 17, 18, 19,
20], [21, 22, 23, 24, 25]], [[26, 27, 28, 29, 30], [31, 32, 33, 34, 35], [36,
37, 38, 39, 40], [41, 42, 43, 44, 45], [46, 47, 48, 49, 50]]], [[[51, 52, 53,
54, 55], [56, 57, 58, 59, 60], [61, 62, 63, 64, 65], [66, 67, 68, 69, 70], [71,
72, 73, 74, 75]], [[76, 77, 78, 79, 80], [81, 82, 83, 84, 85], [86, 87, 88, 89,
90], [91, 92, 93, 94, 95], [96, 97, 98, 99, 100]]]]
```

3 Understanding Tensors: https://www.tensorflow.org/tutorials/quickstart/ https://pytorch.org/

4 A tensor is a multi-dimensional array used primarily in scientific computing and machine learning. In the context of machine learning libraries like TensorFlow or PyTorch, tensors are the primary data structures for building and training models.

5 Tensors are optimized for mathematical operations, making them more efficient for large-scale computations. Tensors can represent data in higher dimensions (1D, 2D, 3D, and beyond), which is essential for deep learning models. Tensors seamlessly integrate with GPUs and other hardware accelerators for faster computations.

5.1 Lists vs Tensors

6 Lists: General-purpose container for storing a collection of items.

7 One-dimensional or multi-dimensional. SUpport Basic operations like indexing, slicing, appending, and removing elements. Can be resized dynamically by adding or removing elements.

8 Tensors: Specialized data structure for numerical computations, primarily used in machine learning and scientific computing. Multi-dimensional arrays with a uniform data type (e.g., integers, floats). Designed for efficient numerical operations, such as matrix multiplication, element-wise operations, and more.

8.1 All elements must be of the same data type. The size is fixed once the tensor is created (though some operations can produce new tensors of different sizes).

Lists: Not optimized for performance in numerical operations. Tensors: Highly optimized for performance, especially with support for hardware accelerations like GPUs.

9 Lists: Built-in Python data structure.

Tensors: Provided by specialized libraries such as TensorFlow, PyTorch, and NumPy. **Lists**: General-purpose programming, storing collections of items, implementing data structures like stacks and queues. Tensors: Machine learning, deep learning, scientific computing, and any domain requiring efficient numerical computations.

9.1 Practice: SOLVE ALL OF THESE EXAERCISES

[8]:
```
#Exercise 1: Create a List of Squares
#Task: Create a list of squares of numbers from 1 to 10.
squares = [x**2 for x in range(1, 11)]
print(squares)
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

[9]:
```
#Exercise 2: Filter Even Numbers
#Task: Create a list of even numbers from 1 to 20.
evens = [x for x in range(1, 21) if x % 2 == 0]
print(evens)
```

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

[10]:
```
#Exercise 3: Extract First Letters
#Task: Given a list of words, create a list of their first letters.
words = ["apple", "banana", "cherry", "date"]
first_letters = [word[0] for word in words]
print(first_letters)
```

```
['a', 'b', 'c', 'd']
```

[11]:
```
#Exercise 4: Create a List of Lengths
#Task: Given a list of words, create a list of their lengths.
words = ["apple", "banana", "cherry", "date"]
lengths = [len(word) for word in words]
print(lengths)
```

```
[5, 6, 6, 4]
```

[12]:
```
#Exercise 5: Multiply Elements by 2
#Task: Given a list of numbers, create a list where each number is multiplied␣
 ↪by 2.
numbers = [1, 2, 3, 4, 5]
doubled = [n * 2 for n in numbers]
print(doubled)
```

```
[2, 4, 6, 8, 10]
```

[13]:
```
#Exercise 6: Filter Strings Containing 'a'
#Task: Given a list of words, create a list of words that contain the letter␣
 ↪'a'.
words = ["apple", "banana", "cherry", "date", "fig"]
```

```
with_a = [word for word in words if 'a' in word]
print(with_a)
```

```
['apple', 'banana', 'date']
```

[14]:
```
#Exercise 7: Flatten a List of Lists
#Task: Given a list of lists, flatten it into a single list.
list_of_lists = [[1, 2, 3], [4, 5], [6, 7, 8]]
flattened = [item for sublist in list_of_lists for item in sublist]
print(flattened)
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

[15]:
```
#Exercise 8: Remove Duplicates
#Task: Given a list with duplicate elements, create a new list without␣
 ↪duplicates.
numbers = [1, 2, 2, 3, 4, 4, 4, 5]
no_duplicates = list(set(numbers))
print(no_duplicates)
```

```
[1, 2, 3, 4, 5]
```

[16]:
```
#Exercise 9: Reverse a List
#Task: Reverse the elements of a list.
numbers = [1, 2, 3, 4, 5]
reversed_list = numbers[::-1]
print(reversed_list)
```

```
[5, 4, 3, 2, 1]
```

9.2 Tensors

9.3 Practice: SOLVE ALL THESE

[17]:
```
#Exercise 1: Create a Tensor of Squares
#Task: Create a tensor of squares of numbers from 1 to 10.

import torch

# Exercise
squares_tensor = torch.tensor([x**2 for x in range(1, 11)])
print(squares_tensor)
```

```
tensor([  1,   4,   9,  16,  25,  36,  49,  64,  81, 100])
```

[18]:
```
#Exercise 2: Filter Even Numbers
#Task: Create a tensor of even numbers from 1 to 20.
evens_tensor = torch.tensor([x for x in range(1, 21) if x % 2 == 0])
```

```
print(evens_tensor)
```

```
tensor([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

[19]:
```
import torch
#Exercise 3: Extract First Letters
#Task: Given a tensor of ASCII values, create a tensor of the corresponding␣
 ↪characters.
ascii_values = torch.tensor([97, 98, 99, 100])  # ASCII for 'a', 'b', 'c', 'd'

# The original code attempts to create a tensor of strings, which causes a␣
 ↪ValueError.
# PyTorch tensors are primarily designed for numerical data.
# To represent characters in a tensor, it's typical to store their ASCII or␣
 ↪Unicode integer values.
# The error `ValueError: too many dimensions 'str'` occurs because `torch.
 ↪tensor` cannot directly
# convert a list of Python strings (like ['a', 'b', 'c', 'd']) into a numerical␣
 ↪tensor.
# To fix this, we create a new tensor directly from the numerical ASCII values.
# We can specify `dtype=torch.uint8` to explicitly indicate that these are byte␣
 ↪values representing characters.
chars_tensor = torch.tensor([x.item() for x in ascii_values], dtype=torch.uint8)

print(chars_tensor)
```

```
tensor([ 97,  98,  99, 100], dtype=torch.uint8)
```

[20]:
```
#Exercise 4: Create a Tensor of Lengths
#Task: Given a list of strings, create a tensor of their lengths.
import torch
words = ["apple", "banana", "cherry", "date"]
lengths_tensor = torch.tensor([len(word) for word in words])
print(lengths_tensor)
```

```
tensor([5, 6, 6, 4])
```

[21]:
```
#Exercise 5: Multiply Elements by 2
#Task: Given a tensor of numbers, create a tensor where each number is␣
 ↪multiplied by 2.
import torch
numbers_tensor = torch.tensor([1, 2, 3, 4, 5])
doubled_tensor = numbers_tensor * 2
print(doubled_tensor)
```

```
tensor([ 2,  4,  6,  8, 10])
```

[22]:
```
#Exercise 6: Filter Strings Containing 'a'
#Task: Given a list of ASCII values, create a tensor of values that correspond␣
 ↪to characters containing 'a'.
import torch
# 'a' is ASCII 97. Keep only the values equal to ord('a').
ascii_values = [97, 98, 97, 100, 97]  # 'a', 'b', 'a', 'd', 'a'
a_values_tensor = torch.tensor([v for v in ascii_values if chr(v) == 'a'])
print(a_values_tensor)
```

```
tensor([97, 97, 97])
```

[23]:
```
#Exercise 7: Flatten a Tensor
#Task: Given a 2D tensor, flatten it into a 1D tensor.
import torch
two_d_tensor = torch.tensor([[1, 2, 3], [4, 5, 6]])
flattened_tensor = two_d_tensor.flatten()
print(flattened_tensor)
```

```
tensor([1, 2, 3, 4, 5, 6])
```

[24]:
```
#Exercise 8: Remove Duplicates
#Task: Given a tensor with duplicate elements, create a new tensor without␣
 ↪duplicates.
import torch
numbers_tensor = torch.tensor([1, 2, 2, 3, 4, 4, 5])
unique_tensor = torch.unique(numbers_tensor)
print(unique_tensor)
```

```
tensor([1, 2, 3, 4, 5])
```

[25]:
```
#Exercise 9: Reverse a Tensor
#Task: Reverse the elements of a 1D tensor.
import torch
numbers_tensor = torch.tensor([1, 2, 3, 4, 5])
reversed_tensor = torch.flip(numbers_tensor, dims=[0])
print(reversed_tensor)
```

```
tensor([5, 4, 3, 2, 1])
```

9.4 Tuples https://www.w3schools.com/python/python_tuples.asp

9.5 Practice: SOLVE this all

[26]:
```
#Exercise 1: Create a Tuple of Squares
#Task: Create a tuple of squares of numbers from 1 to 10.
squares_tuple = tuple(x**2 for x in range(1, 11))
print(squares_tuple)
```

```
(1, 4, 9, 16, 25, 36, 49, 64, 81, 100)
```

[27]:
```
#Exercise 2: Filter Even Numbers
#Task: Create a tuple of even numbers from 1 to 20.
evens_tuple = tuple(x for x in range(1, 21) if x % 2 == 0)
print(evens_tuple)
```

```
(2, 4, 6, 8, 10, 12, 14, 16, 18, 20)
```

[28]:
```
#Exercise 3: Extract First Letters
#Task: Given a list of words, create a tuple of their first letters.
words = ["apple", "banana", "cherry", "date"]
first_letters_tuple = tuple(word[0] for word in words)
print(first_letters_tuple)
```

```
('a', 'b', 'c', 'd')
```

[29]:
```
#Create a Tuple of Lengths
#Task: Given a list of words, create a tuple of their lengths.
words = ["apple", "banana", "cherry", "date"]
lengths_tuple = tuple(len(word) for word in words)
print(lengths_tuple)
```

```
(5, 6, 6, 4)
```

[30]:
```
#Exercise 5: Multiply Elements by 2
#Task: Given a list of numbers, create a tuple where each number is multiplied␣
 ↪by 2.
numbers = [1, 2, 3, 4, 5]
doubled_tuple = tuple(n * 2 for n in numbers)
print(doubled_tuple)
```

```
(2, 4, 6, 8, 10)
```

[31]:
```
#Exercise 6: Filter Strings Containing 'a'
#Task: Given a list of words, create a tuple of words that contain the letter␣
 ↪'a'.
words = ["apple", "banana", "cherry", "date", "fig"]
with_a_tuple = tuple(word for word in words if 'a' in word)
print(with_a_tuple)
```

```
('apple', 'banana', 'date')
```

[32]:
```
#Exercise 7: Create a Tuple of Tuples
#Task: Create a tuple of tuples, where each inner tuple contains a number and its square, for numbers from 1 to 5.
number_squares = tuple((x, x**2) for x in range(1, 6))
print(number_squares)
```

```
((1, 1), (2, 4), (3, 9), (4, 16), (5, 25))
```

[33]:
```
#Exercise 8: Flatten a Tuple of Tuples
#Task: Given a tuple of tuples, flatten it into a single tuple.
tuple_of_tuples = ((1, 2), (3, 4), (5, 6))
flattened_tuple = tuple(item for inner in tuple_of_tuples for item in inner)
print(flattened_tuple)
```

```
(1, 2, 3, 4, 5, 6)
```

[34]:
```
#Exercise 9: Remove Duplicates
#Task: Given a list with duplicate elements, create a tuple without duplicates.
numbers = [1, 2, 2, 3, 4, 4, 5]
no_duplicates_tuple = tuple(set(numbers))
print(no_duplicates_tuple)
```

```
(1, 2, 3, 4, 5)
```

[35]:
```
#Exercise 10: Reverse a Tuple
#Task: Reverse the elements of a tuple.
my_tuple = (1, 2, 3, 4, 5)
reversed_tuple = my_tuple[::-1]
print(reversed_tuple)
```

```
(5, 4, 3, 2, 1)
```

9.6 Dictionaries https://www.w3schools.com/python/python_dictionaries.asp

10 Practice:

[36]:
```
#RUN IT
# Import necessary libraries
import random
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

[37]:
```python
##add your name to this dictionary with any grade you like
# Creating and manipulating dictionaries
student_grades = {'Alice': 'A', 'Bob': 'B', 'Charlie': 'C', 'Vikram': 'A'}
print("student_grades:", student_grades)

# Accessing & counting elements
print("Bob's grade:", student_grades['Bob'])
print("len(student_grades)", len(student_grades))

# Adding and modifying elements
student_grades['David'] = 'B+'
student_grades['Alice'] = 'A+'
print("\nUpdated dictionary:", student_grades)
print("len(student_grades)", len(student_grades))

# Deleting an element
del student_grades['Charlie'] #del is a keyword that serves the purpose of␣
 ↪deleting objects
print("\nstudent_grades after deletion:", student_grades)
print("\nlen(student_grades)", len(student_grades))

# Iterating over dictionary
for student, grade in student_grades.items():
    print(f"{student}: {grade}")
```

```
student_grades: {'Alice': 'A', 'Bob': 'B', 'Charlie': 'C', 'Vikram': 'A'}
Bob's grade: B
len(student_grades) 4

Updated dictionary: {'Alice': 'A+', 'Bob': 'B', 'Charlie': 'C', 'Vikram': 'A',
'David': 'B+'}
len(student_grades) 5

student_grades after deletion: {'Alice': 'A+', 'Bob': 'B', 'Vikram': 'A',
'David': 'B+'}

len(student_grades) 4
Alice: A+
Bob: B
Vikram: A
David: B+
```

Dictionaries (dict) allow fast lookups Java Equivalent: HashMaps and TreeMaps. **Use Cases:** Storing configuration data. Counting word frequencies. Representing relationships between entities.

Similar code in Java

[38]:
```
#import java.util.HashMap;
#import java.util.Map;

#public class Main {
#    public static void main(String[] args) {
#        Map<String, String> studentGrades = new HashMap<>();
#        studentGrades.put("Alice", "A");
#        studentGrades.put("Bob", "B");
#        studentGrades.put("Charlie", "C");
#
#        // Iterating over entries in the map
#        for (Map.Entry<String, String> entry : studentGrades.entrySet()) {
#            String student = entry.getKey();
#            String grade = entry.getValue();
#            System.out.println(student + ": " + grade);
#        }
#    }
#}
```

11 Practice: Count the occurrences of each word in a sentence using a dictionary.

[39]:
```
#RUN IT
# Example using dict() function
my_dict = dict(name='Alice', age=30, city='New York')
#add your name to this dictionary, print length and content
my_dict['student_name'] = 'Vikram'
print("Length:", len(my_dict))
print("Content:", my_dict)
```

```
Length: 4
Content: {'name': 'Alice', 'age': 30, 'city': 'New York', 'student_name':
'Vikram'}
```

[40]:
```
#RUN IT
sentence = "this is a test this is only a test"
word_counts = {}

words = sentence.split()
print(words)

for word in words: #The .split() method breaks the sentence into individual␣
 ↪words, based on spaces.
    if word in word_counts:
        word_counts[word] += 1
```

```
        print(word_counts)
    else:
        word_counts[word] = 1
        print(word_counts)
print("Word counts:", word_counts)
```

```
['this', 'is', 'a', 'test', 'this', 'is', 'only', 'a', 'test']
{'this': 1}
{'this': 1, 'is': 1}
{'this': 1, 'is': 1, 'a': 1}
{'this': 1, 'is': 1, 'a': 1, 'test': 1}
{'this': 2, 'is': 1, 'a': 1, 'test': 1}
{'this': 2, 'is': 2, 'a': 1, 'test': 1}
{'this': 2, 'is': 2, 'a': 1, 'test': 1, 'only': 1}
{'this': 2, 'is': 2, 'a': 2, 'test': 1, 'only': 1}
{'this': 2, 'is': 2, 'a': 2, 'test': 2, 'only': 1}
Word counts: {'this': 2, 'is': 2, 'a': 2, 'test': 2, 'only': 1}
```

11.1 Class Challenge

[41]:
```
import nltk
from nltk.corpus import stopwords

# Download stopwords
nltk.download('stopwords')

##CREATE A FILE WITH A LONG TEXT, mount your drive to connect to it
# Read the text from the file
# In Colab you can mount Drive with:
from google.colab import drive
drive.mount('/content/drive')
with open('/content/drive/MyDrive/my_text.txt', 'r') as f:
      text = f.read()
# For a self-contained example we define the text directly:
#text = (
#    "This is a sample text. This text is only a sample. "
#   "We are counting how many times each word appears in this text. "
#    "The text is simple and the words repeat so we can see the counts."
#)

# Create a Function to count word occurrences
def count_words(text):
    word_counts = {}
    for word in text.lower().split():
        word = word.strip('.,!?;:"()')  # remove punctuation
        if word == '':
            continue
```

```
        if word in word_counts:
            word_counts[word] += 1
        else:
            word_counts[word] = 1
    return word_counts

# Create a word count dictionary
word_counts = count_words(text)
print("Word counts:", word_counts)

# Import stopwords
stop_words = set(stopwords.words('english'))

# Function to count stopword occurrences
def count_stopwords(word_counts, stop_words):
    stopword_counts = {}
    for word, count in word_counts.items():
        if word in stop_words:
            stopword_counts[word] = count
    return stopword_counts

# Create a stopword presence dictionary
stopword_counts = count_stopwords(word_counts, stop_words)
print("Stopword counts:", stopword_counts)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
Word counts: {'{\\rtf1\\ansi\\ansicpg1252\\cocoartf2870': 1,
'\\cocoatextscaling0\\cocoaplatform0{\\fonttbl\\f0\\froman\\fcharset0': 1,
'times-roman;}': 1,
'{\\colortbl;\\red255\\green255\\blue255;\\red0\\green0\\blue0;}': 1,
'{\\*\\expandedcolortbl;;\\cssrgb\\c0\\c0\\c0;}': 1,
'\\margl1440\\margr1440\\vieww11520\\viewh8400\\viewkind0': 1, '\\deftab720': 1,
'\\pard\\pardeftab720\\partightenfactor0': 1, '\\f0\\fs24': 1, '\\cf0': 1,
'\\expnd0\\expndtw0\\kerning0': 1, '\\outl0\\strokewidth0': 1, '\\strokec2': 1,
'however': 1, 'little': 1, 'known': 1, 'the': 22, 'feelings': 1, 'or': 3,
'views': 1, 'of': 20, 'such': 1, 'a': 10, 'man': 1, 'may': 1, 'be': 1, 'on': 4,
'his': 1, 'first': 1, 'entering': 1, 'neighbourhood': 1, 'this': 1, 'truth': 1,
'is': 2, 'so': 2, 'well': 1, 'fixed': 1, 'in': 4, 'minds': 1, 'surrounding': 1,
'families': 1, 'that': 3, 'he': 1, 'considered': 1, 'rightful': 1, 'property':
1, 'some': 2, 'one': 1, 'other': 2, 'their': 1, 'daughters.\\': 1, 'it': 11,
'was': 12, 'best': 1, 'times': 2, 'worst': 1, 'age': 2, 'wisdom': 1,
'foolishness': 1, 'epoch': 2, 'belief': 1, 'incredulity': 1, 'season': 2,
'light': 1, 'darkness': 1, 'spring': 1, 'hope': 1, 'winter': 1, 'despair': 1,
'we': 4, 'had': 2, 'everything': 1, 'before': 2, 'us': 2, 'nothing': 1, 'were':
```

```
5, 'all': 2, 'going': 2, 'direct': 2, 'to': 2, 'heaven': 1, "way\\'97in": 1,
'short': 1, 'period': 2, 'far': 1, 'like': 1, 'present': 1, 'its': 2,
'noisiest': 1, 'authorities': 1, 'insisted': 1, 'being': 1, 'received': 1,
'for': 3, 'good': 1, 'evil': 1, 'superlative': 1, 'degree': 1, 'comparison': 1,
'only.\\': 1, 'there': 2, 'king': 2, 'with': 4, 'large': 2, 'jaw': 2, 'and': 3,
'queen': 2, 'plain': 1, 'face': 2, 'throne': 2, 'england': 1, 'fair': 1,
'france': 1, 'both': 1, 'countries': 1, 'clearer': 1, 'than': 1, 'crystal': 1,
'lords': 1, 'state': 1, 'preserves': 1, 'loaves': 1, 'fishes': 1, 'things': 1,
'general': 1, 'settled': 1, 'ever.}': 1}
Stopword counts: {'the': 22, 'or': 3, 'of': 20, 'such': 1, 'a': 10, 'be': 1,
'on': 4, 'his': 1, 'this': 1, 'is': 2, 'so': 2, 'in': 4, 'that': 3, 'he': 1,
'some': 2, 'other': 2, 'their': 1, 'it': 11, 'was': 12, 'we': 4, 'had': 2,
'before': 2, 'were': 5, 'all': 2, 'to': 2, 'its': 2, 'being': 1, 'for': 3,
'there': 2, 'with': 4, 'and': 3, 'both': 1, 'than': 1}
```

11.2 Practice:

[42]:
```python
#RUN IT LINE BY LINE, UNCOMMENT AS YOU TRACE
# Creating and manipulating sets
fruits = {'apple', 'banana', 'cherry'}
print("Initial set:", fruits)

# Adding elements
fruits.add('orange')
print("Set after adding an element:", fruits)

# Removing elements
fruits.remove('banana')
print("Set after removing an element:", fruits)

# Set operations: union, intersection, difference
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
print("Union:", set1 | set2)
print("Intersection:", set1 & set2)
print("Difference:", set1 - set2)

#Find the common elements in two lists of words.

list1 = ['apple', 'banana', 'cherry', 'date']
list2 = ['cherry', 'date', 'elderberry', 'fig']
```

```
common_elements = set(list1) & set(list2)
print("Common elements:", common_elements)
```

```
Initial set: {'banana', 'apple', 'cherry'}
Set after adding an element: {'orange', 'banana', 'apple', 'cherry'}
Set after removing an element: {'orange', 'apple', 'cherry'}
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference: {1, 2}
Common elements: {'date', 'cherry'}
```

12 Dictionary comprehension: an expression that reads a sequence of input elements and uses those input elements to produce a dictionary

13 Practice!

[43]:
```
#RUN IT, TRACE THIS CODE LINE BY LINE, FILL IN THE MISSING CODE
# Start from a known dictionary so the cell runs on its own
student_grades = {'Alice': 'A+', 'Bob': 'B', 'David': 'B+'}

# Use the get method to access Alice's grade
alice_grade = student_grades.get('Alice')
print("Alice's grade (using get):", alice_grade)

# Use the get method with a default value for a non-existent key
eve_grade = student_grades.get('Eve', 'N/A')
print("Eve's grade (using get with default):", eve_grade)

# Update multiple elements at once
student_grades.update({'Bob': 'A', 'Frank': 'C'})
print("Dictionary after bulk update:", student_grades)

# Remove an element and return its value with pop() method
removed_grade = student_grades.pop('David')
print("Removed David's grade:", removed_grade)
print("Dictionary after removing David:", student_grades)

# Clear all elements from the dictionary
student_grades.clear()
print("Dictionary after clearing all elements:", student_grades)
```

```python
# Re-adding elements to demonstrate further modifications
student_grades['Grace'] = 'A'
student_grades['Heidi'] = 'B'
student_grades['Ivan'] = 'C'
print("Dictionary after re-adding elements:", student_grades)

# Demonstrate dictionary methods: keys, values, and items
print("Keys:", student_grades.keys())
print("Values:", student_grades.values())
print("Items:", student_grades.items())

# Merge two dictionaries using update
additional_grades = {'Jack': 'B+', 'Kathy': 'A-'}
student_grades.update(additional_grades)
print("Dictionary after merging with additional grades:", student_grades)

# Create a dictionary using dictionary comprehension - we applied an expression
squared_numbers = {x: x**2 for x in range(5)}
print("Dictionary with squared numbers:", squared_numbers)

# Create a dictionary with square root of numbers in range up to 10
square_roots = {x: x**0.5 for x in range(10)}
print("Square roots:", square_roots)

# Count the occurrences of each character in a string using a dictionary
def count_characters(text):
    char_counts = {}
    for char in text:
        char_counts[char] = char_counts.get(char, 0) + 1
    return char_counts

text = "hello world"
print("Character counts:", count_characters(text))

# Merge two dictionaries representing the grades of students from two different␣
 ↪classes
#add your name to this code snippet as a part of the dictionary
class1_grades = {'Alice': 'A', 'Bob': 'B', 'Vikram': 'A'}
class2_grades = {'Charlie': 'C', 'David': 'B+'}

combined_grades = class1_grades.copy()
combined_grades.update(class2_grades)
print("Combined grades:", combined_grades)
```

```
Alice's grade (using get): A+
Eve's grade (using get with default): N/A
Dictionary after bulk update: {'Alice': 'A+', 'Bob': 'A', 'David': 'B+',
```

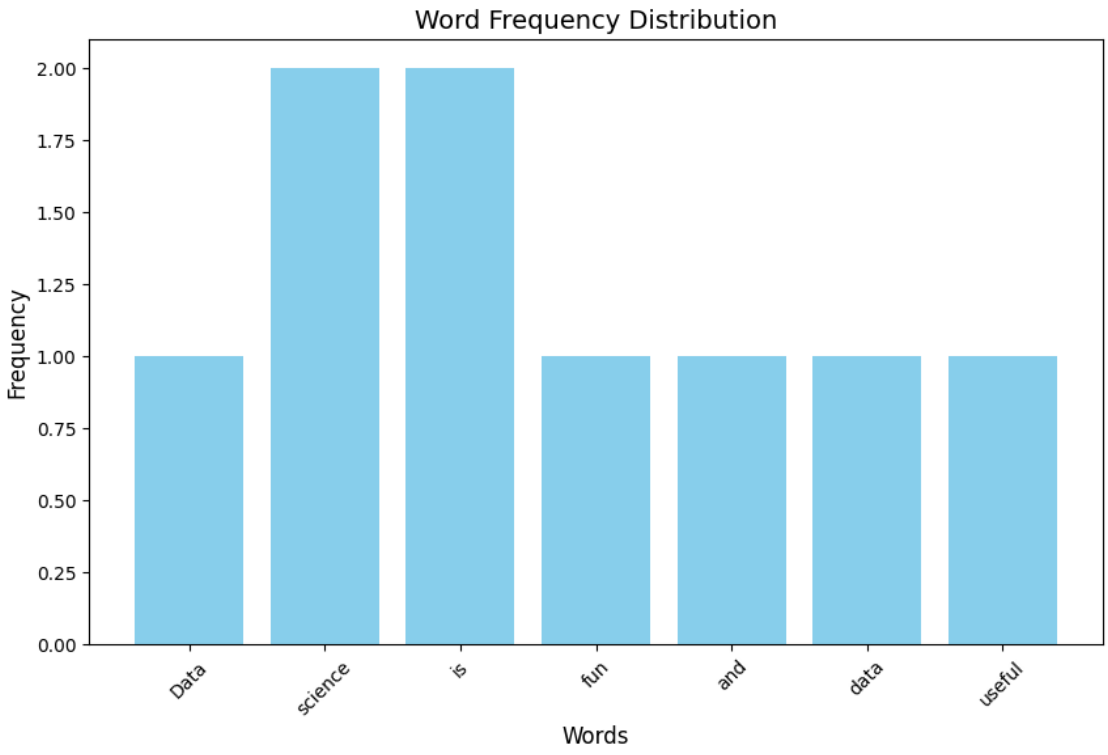
```
'Frank': 'C'}
Removed David's grade: B+
Dictionary after removing David: {'Alice': 'A+', 'Bob': 'A', 'Frank': 'C'}
Dictionary after clearing all elements: {}
Dictionary after re-adding elements: {'Grace': 'A', 'Heidi': 'B', 'Ivan': 'C'}
Keys: dict_keys(['Grace', 'Heidi', 'Ivan'])
Values: dict_values(['A', 'B', 'C'])
Items: dict_items([('Grace', 'A'), ('Heidi', 'B'), ('Ivan', 'C')])
Dictionary after merging with additional grades: {'Grace': 'A', 'Heidi': 'B',
'Ivan': 'C', 'Jack': 'B+', 'Kathy': 'A-'}
Dictionary with squared numbers: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
Square roots: {0: 0.0, 1: 1.0, 2: 1.4142135623730951, 3: 1.7320508075688772, 4:
2.0, 5: 2.23606797749979, 6: 2.449489742783178, 7: 2.6457513110645907, 8:
2.8284271247461903, 9: 3.0}
Character counts: {'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd':
1}
Combined grades: {'Alice': 'A', 'Bob': 'B', 'Vikram': 'A', 'Charlie': 'C',
'David': 'B+'}
```

14 Practice: the Summary

[44]:
```python
#RUN IT
# Import necessary libraries
import random
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Dictionaries
print("### Dictionaries ###")

# Explanation
print("Dictionaries are key-value pairs, unordered, fast lookups. Similar to␣
 ↪Java's HashMaps and TreeMaps.")

# Similarities and Differences with Java
print("""
In Java, dictionaries are implemented as HashMaps or TreeMaps.
- Both Python dictionaries and Java HashMaps provide constant-time complexity␣
 ↪for insertions and lookups.
- Both are unordered collections.
- Python dictionaries can store heterogeneous data types, while Java's HashMap␣
 ↪typically stores objects of the same type.
- Java TreeMap maintains a sorted order, unlike Python dictionaries.
""")
```

```
# Example 1: Creating and manipulating a dictionary
print("\nCreating and manipulating a dictionary")
#add your name to this dictionary
student_scores = {
    'Alice': 85,
    'Bob': 92,
    'Charlie': 78,
    'David': 90
}
print(student_scores)

# Accessing elements
print("Alice's score:", student_scores['Alice'])

# Adding a new key-value pair
student_scores['Eve'] = 88
print("Updated dictionary:", student_scores)

# Challenge 1: Count word occurrences in a sentence
print("\nChallenge 1: Count word occurrences in a sentence")
sentence = "Data science is fun and data science is useful"
word_counts = {}
words = sentence.split()
for word in words:
    if word in word_counts:
        word_counts[word] += 1
    else:
        word_counts[word] = 1
print("Word counts:", word_counts)

#Let's visualize our Dictionary
# Prepare data for plotting
words = list(word_counts.keys())
frequencies = list(word_counts.values())

# Create the bar chart
plt.figure(figsize=(10, 6))  # Optional: Adjust figure size
plt.bar(words, frequencies, color='skyblue')

# Add labels and title
plt.xlabel('Words', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.title('Word Frequency Distribution', fontsize=14)
plt.xticks(rotation=45)  # Rotate x-axis labels for readability

# Show the plot
plt.show()
```

```
### Dictionaries ###
Dictionaries are key-value pairs, unordered, fast lookups. Similar to Java's
HashMaps and TreeMaps.

In Java, dictionaries are implemented as HashMaps or TreeMaps.
- Both Python dictionaries and Java HashMaps provide constant-time complexity
for insertions and lookups.
- Both are unordered collections.
- Python dictionaries can store heterogeneous data types, while Java's HashMap
typically stores objects of the same type.
- Java TreeMap maintains a sorted order, unlike Python dictionaries.


Creating and manipulating a dictionary
{'Alice': 85, 'Bob': 92, 'Charlie': 78, 'David': 90}
Alice's score: 85
Updated dictionary: {'Alice': 85, 'Bob': 92, 'Charlie': 78, 'David': 90, 'Eve':
88}

Challenge 1: Count word occurrences in a sentence
Word counts: {'Data': 1, 'science': 2, 'is': 2, 'fun': 1, 'and': 1, 'data': 1,
'useful': 1}
```



14.1 Class Challenge

15 Understand the game "Tresure Hunt"

[45]:
```
#RUN IT
import random

#Game 1: Treasure Hunt and Creating a Dictionary to Store Player Points

# Creating a dictionary to store player points
#add your name to this dictionary
player_points = {'Alice': 0, 'Bob': 0, 'Charlie': 0, 'Vikram': 0}

# Function to simulate finding a treasure
def find_treasure(player):
    import random
    points = random.randint(1, 100)
    player_points[player] += points
    print(f"{player} found a treasure worth {points} points!")

#Running the Game
find_treasure('Alice')
find_treasure('Bob')
find_treasure('Charlie')
find_treasure('Vikram')

# Display the updated points
print("\nUpdated player points:", player_points)

# Determining the winners
max_points = max(player_points.values())
winners = [player for player, points in player_points.items() if points ==␣
 ↪max_points]
print(f"\nThe winner(s): {', '.join(winners)} with {max_points} points!")
```

```
Alice found a treasure worth 46 points!
Bob found a treasure worth 39 points!
Charlie found a treasure worth 97 points!
Vikram found a treasure worth 26 points!

Updated player points: {'Alice': 46, 'Bob': 39, 'Charlie': 97, 'Vikram': 26}

The winner(s): Charlie with 97 points!
```

16 Implement the game "Rock-Paper-Scissors"

[46]:
```
import random

# Creating a Dictionary to Store Player Points
#add your name to this dictionary
player_points = {'Alice': 0, 'Bob': 0, 'Charlie': 0, 'Vikram': 0}

# Function: Playing Rock-Paper-Scissors
# The function play_rps takes two players as input.
# Each player randomly chooses rock, paper, or scissors.
# The function determines the winner and updates their points.
def play_rps(player1, player2):
    choices = ['rock', 'paper', 'scissors']
    p1_choice = random.choice(choices)
    p2_choice = random.choice(choices)

    print(f"{player1} chose {p1_choice}")
    print(f"{player2} chose {p2_choice}")

    # Logic to determine the winner and update points
    beats = {'rock': 'scissors', 'paper': 'rock', 'scissors': 'paper'}

    if p1_choice == p2_choice:
        print("It's a tie!")
    elif beats[p1_choice] == p2_choice:
        player_points[player1] += 1
        print(f"{player1} wins this round!")
    else:
        player_points[player2] += 1
        print(f"{player2} wins this round!")
    print()

# Players play rock-paper-scissors
play_rps('Alice', 'Bob')
play_rps('Charlie', 'Vikram')
play_rps('Alice', 'Vikram')

# Display the updated points
print("Updated player points:", player_points)

# Task 2: Determining the Winners
max_points = max(player_points.values())
winners = [player for player, points in player_points.items() if points ==␣
 ↪max_points]
print(f"The winner(s): {', '.join(winners)} with {max_points} points!")
```

```
Alice chose scissors
```

```
Bob chose scissors
It's a tie!

Charlie chose paper
Vikram chose rock
Charlie wins this round!

Alice chose scissors
Vikram chose rock
Vikram wins this round!

Updated player points: {'Alice': 0, 'Bob': 0, 'Charlie': 1, 'Vikram': 1}
The winner(s): Charlie, Vikram with 1 points!
```

16.1 List vs. Dictionary Fibonacci Time Competition

[47]:
```python
import time
# Function to calculate Fibonacci using a list (bottom-up approach)
# Implement Fibonacci calculation using a list
def fibonacci_list(n):
    fib_list = [0, 1]
    for i in range(2, n+1):
        fib_list.append(fib_list[i-1] + fib_list[i-2])
    return fib_list[n]

# Function to calculate Fibonacci using a dictionary (bottom-up approach)
# Implement Fibonacci calculation using a dictionary
def fibonacci_dict(n):
    fib_dict = {0: 0, 1: 1}
    for i in range(2, n+1):
        fib_dict[i] = fib_dict[i-1] + fib_dict[i-2]
    return fib_dict[n]

# Function to measure execution time of Fibonacci calculation
def measure_time(fibonacci_func, n):
    start_time = time.time()
    result = fibonacci_func(n)
    end_time = time.time()
    return end_time - start_time, result

# Number for which to calculate Fibonacci
n = 10000

# Measure and compare execution times
# Analyze THIS CODE: Call measure_time for both fibonacci_list and␣
 ↪fibonacci_dict and print the times and results
list_time, list_result = measure_time(fibonacci_list, n)
```

```
dict_time, dict_result = measure_time(fibonacci_dict, n)

print(f"Fibonacci number (list) for n={n}: {list_result}")
print(f"Execution time (list): {list_time:.10f} seconds")

print(f"Fibonacci number (dictionary) for n={n}: {dict_result}")
print(f"Execution time (dictionary): {dict_time:.10f} seconds")
```

```
Fibonacci number (list) for n=10000: 336447648764317832666216120051075433103021484606800639065647699746800814421666623681555955136337340255820653326808361593737347904838652682630408924630564318873545443695598274916066020998841839338646527313000888302692356736131351175792974378544137521305205043477016022647583189065278908855154366159582987279682987510631200575428783453215515103870818298969791613127856265033195487140214287532698187962046936097879900350962302291026368131493195275630227837628441540360584402572114334961180023091208287046088923962328835461505776583271252546093591128203925285393434620904245248929403901706233888991085841065183173360437470737908552631764325733993712871937587746897479926305837065742830161637408969178426378624212835258112820516370298089332099905707920064367426202389783111470054074998459250360633560933883831923386783056136435351892133279732908133732642652633989763922723407882928177953580570993691049175470808931841056146322338217465637321248226383092103297701648054726243842374862411453093812206564914032751086643394517512161526545361333111314042436854805106765843493523836959653428071768775328348234345557366719731392746273629108210679280784718035329131176778924659089938635459327894523777674406192240337638674004021330343297496902028328145933418826817683893072003634795623117103101291953169794607632737589253530772552375943788434504067715555779056450443016640119462580972216729758615026968443146952034614932291105970676243268515992834709891284706740862008587135016260312071903172086094081298321581077282076353186624611278245537208532365305775956430072517744315051539600905168603220349163222640885248852433158051534849622434848299380905070483482449327453732624567755879089187190803662058009594743150052402532709746995318770724376825907419939632265984147498193609285223945039707165443156421328157688908058783183404917434556270520223564846495196112460268313970975069382648706613264507665074611512677522748621598642530711298441182622661057163515069260029861704945425047491378115154139941550671256271197133252763631939606902895650288268608362241082050562430701794976171121233066073310059947366875Execution time (list): 0.0044414997 secondsFibonacci number (dictionary) for n=10000: 336447648764317832666216120051075433103021484606800639065647699746800814421666623681555955136337340255820653326808361593737347904838652682630408924630564318873545443695598274916066020998841839338646527313000888302692356736131351175792974378544137521305205043477016022647583189065278908855154366159582987279682987510631200575428783453215515103870818298969791613127856265033195487140214287532698187962046936097879900350962302291026368131493195275630227837628441540360584402572114334961180023091208287046088923962328835461505776583271252546093591128203925285393434620904245248929403901706233888991085841065183173360437470737908552631764325733993712871937587746897479926305837065742830161637408969178426378624212835258112820516370298089332099905707920064367426202389783111470054074998459250360633560933883831923386783056136435351892133279732908133732642652633989763922723407882928177953580570993691049175470808931841056146
```

```
32233821746563732124822638309210329770164805472624384237486241145309381220656491
40327510866433945175121615265453613331113140424368548051067658434935238369596534
28071768775328348234345557366719731392746273629108210679280784718035329131176778
92465908993863545932789452377767440619224033763867400402133034329749690202832814
59334188268176838930720036347956231171031012919531697946076327375892535307725523
75943788434504067715555779056450443016640119462580972216729758615026968443146952
03461493229110597067624326851599283470989128470674086200858713501626031207190317
20860940812983215810772820763531866246112782455372085323653057759564300725177443
15051539600905168603220349163222640885248852433158051534849622434848299380905070
48348244932745373262456775587908918719080366205800959474315005240253270974699531
87707243768259074199396322659841474981936092852239450397071654431564213281576889
08058783183404917434556270520223564846495196112460268313970975069382648706613264
50766507461151267752274862159864253071129844118262266105716351506926002986170494
54250474913781151541399415506712562711971332527636319396069028956502882686083622
41082050562430701794976171121233066073310059947366875
Execution time (dictionary): 0.0082161427 seconds
```

16.2 https://www.w3schools.com/python/python_sets.asp

17 Practice:

[48]:
```
# Sets
print("\n### Sets ###")

# Explanation
print("Sets are unordered collections of unique elements, efficient membership␣
 ↪testing and set operations. Similar to Java's HashSet and TreeSet.")

# Similarities and Differences with Java
print("""
In Java, sets are implemented as HashSet or TreeSet.
- Both Python sets and Java HashSets provide constant-time complexity for␣
 ↪insertions and lookups.
- Both are unordered collections and store unique elements.
- Java TreeSet maintains a sorted order, unlike Python sets.
- Python sets are more flexible in terms of element types compared to Java's␣
 ↪typed sets.
""")

# Example 1: Creating and manipulating a set
print("\nExample 1: Creating and manipulating a set")
unique_numbers = {1, 2, 3, 4, 5}
```

```
print("Original set:", unique_numbers)

# Adding elements
unique_numbers.add(6)
print("Updated set:", unique_numbers)

# Example 2: Find common elements in two lists
print("\nChallenge 2: Find common elements in two lists")
list1 = [1, 2, 3, 4, 5]
list2 = [4, 5, 6, 7, 8]
common_elements = set(list1) & set(list2)
print("Common elements:", common_elements)
```

```
### Sets ###
Sets are unordered collections of unique elements, efficient membership testing
and set operations. Similar to Java's HashSet and TreeSet.

In Java, sets are implemented as HashSet or TreeSet.
- Both Python sets and Java HashSets provide constant-time complexity for
insertions and lookups.
- Both are unordered collections and store unique elements.
- Java TreeSet maintains a sorted order, unlike Python sets.
- Python sets are more flexible in terms of element types compared to Java's
typed sets.


Example 1: Creating and manipulating a set
Original set: {1, 2, 3, 4, 5}
Updated set: {1, 2, 3, 4, 5, 6}

Challenge 2: Find common elements in two lists
Common elements: {4, 5}
```

Use cases: Removing **duplicates** from a list. Finding unique elements in multiple lists. Performing mathematical set operations.

18 Practice:

[49]:
```
# Creating and manipulating sets
fruits = {'apple', 'banana', 'cherry'}
print("Initial set:", fruits)

# Adding elements
fruits.add('orange')
print("Set after adding an element:", fruits)
```

```
# Removing elements
fruits.remove('banana')
print("Set after removing an element:", fruits)

# Set operations: union, intersection, difference
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
print("Union:", set1 | set2)
print("Intersection:", set1 & set2)
print("Difference:", set1 - set2)

#Exercise:
#Find the common elements in two lists of words.
list1 = ['apple', 'banana', 'cherry', 'date']
list2 = ['cherry', 'date', 'elderberry', 'fig']
common_elements = set(list1) & set(list2)
print("Common elements:", common_elements)
```

```
Initial set: {'banana', 'apple', 'cherry'}
Set after adding an element: {'orange', 'banana', 'apple', 'cherry'}
Set after removing an element: {'orange', 'apple', 'cherry'}
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference: {1, 2}
Common elements: {'date', 'cherry'}
```

19 List vs. Set Lookup Competition

[50]:
```
#Creating a Large List and Set of Random Numbers
import random
import time

large_list = [random.randint(0, 100000) for _ in range(100000)]
large_set = set(large_list)

# Analyze THIS CODE: Measure the time taken to lookup a number
# Function to measure lookup time in a list
def measure_list_lookup(lst, num):
    start_time = time.time()
    result = num in lst
    end_time = time.time()
    return end_time - start_time

# Function to measure lookup time in a set
def measure_set_lookup(st, num):
    start_time = time.time()
    result = num in st
```

```
    end_time = time.time()
    return end_time - start_time


#Comparing Lookup Times
# Number to lookup
lookup_number = random.randint(0, 10000)

# Measure and compare lookup times
list_lookup_time = measure_list_lookup(large_list, lookup_number)
set_lookup_time = measure_set_lookup(large_set, lookup_number)

print(f"Lookup time in list: {list_lookup_time:.10f} seconds")
print(f"Lookup time in set: {set_lookup_time:.10f} seconds")
```

```
Lookup time in list: 0.0006916523 seconds
Lookup time in set: 0.0000004768 seconds
```

[50]:

19.1 Practice:

[51]:
```
#Student Enrollment System
#Create a set to manage student enrollments and perform various set operations.

# Create sets to store students in different courses
#add your name to this set
python_students = {'Alice', 'Bob', 'Charlie', 'Vikram'}
data_science_students = {'Bob', 'David', 'Eve', 'Vikram'}
```

```python
# Adding a new student to the Python course
python_students.add('Grace')
print("Python students after adding Grace:", python_students)

# Removing a student from the Data Science course
data_science_students.remove('David')
print("Data Science students after removing David:", data_science_students)

# Find students enrolled in both courses (intersection)
both_courses = python_students & data_science_students
print("Students in both courses:", both_courses)

# Find students only in the Python course (difference)
only_python = python_students - data_science_students
print("Students only in Python:", only_python)

# Find all students in either course (union)
all_students = python_students | data_science_students
print("All students:", all_students)
```

```
Python students after adding Grace: {'Grace', 'Bob', 'Vikram', 'Charlie',
'Alice'}
Data Science students after removing David: {'Vikram', 'Eve', 'Bob'}
Students in both courses: {'Vikram', 'Bob'}
Students only in Python: {'Charlie', 'Grace', 'Alice'}
All students: {'Charlie', 'Eve', 'Grace', 'Bob', 'Vikram', 'Alice'}
```

19.2 Practice:

[52]:
```python
# Unique Words in a Text
#Create a function to find all unique words in a given text.
#Call this function on your test text
def unique_words(text):
    words = text.lower().split()
    cleaned = [w.strip('.,!?;:"()') for w in words]
    return set(w for w in cleaned if w != '')

test_text = "This is a test This test is only a sample test"
print("Unique words:", unique_words(test_text))
print("Number of unique words:", len(unique_words(test_text)))
```

```
Unique words: {'only', 'this', 'is', 'test', 'sample', 'a'}
Number of unique words: 6
```

19.3 Practice:

[53]:
```
#Social Media Friends
#Simulate a social media application where users have different sets of friends␣
 ↪and perform set operations to find mutual friends, unique friends, etc.

# Create sets to store friends of different users
#add your name to at least one set
alice_friends = {'Bob', 'Charlie', 'David', 'Vikram'}
bob_friends = {'Alice', 'Eve', 'Charlie', 'Vikram'}

# Find mutual friends (intersection)
mutual_friends = alice_friends & bob_friends
print("Mutual friends:", mutual_friends)

# Find unique friends of Alice (difference)
alice_unique = alice_friends - bob_friends
print("Friends unique to Alice:", alice_unique)

# Find all friends (union)
all_friends = alice_friends | bob_friends
print("All friends:", all_friends)
```

```
Mutual friends: {'Charlie', 'Vikram'}
Friends unique to Alice: {'David', 'Bob'}
All friends: {'David', 'Bob', 'Vikram', 'Charlie', 'Eve', 'Alice'}
```

19.4 Bonus part: Understanding DataFrames in Python

A **DataFrame** is a **two-dimensional**, **size-mutable**, and potentially heterogeneous **tabular data structure** with **labeled axes** (rows and columns) provided by the **Pandas** library. It is one of the most commonly used data structures for data manipulation and analysis in Python.

DataFrames store data in a **table format**, similar to a spreadsheet or SQL table. Columns can contain **different types of data** (e.g., integers, floats, strings). Rows and columns have **labels**, making it easier to access and manipulate data. DataFrames ***can be modified*** by adding or removing rows and columns.

[54]:
```
#Exercise 1: Create a DataFrame
import pandas as pd

# Exercise
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [24, 27, 22, 32],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}
df = pd.DataFrame(data)
print(df)
```

```
      Name  Age         City
0    Alice   24     New York
1      Bob   27  Los Angeles
2  Charlie   22      Chicago
3    David   32      Houston
```

[55]:
```
#Exercise 2: Select a Column 'Name' from the DataFrame.
print(df['Name'])
```

```
0      Alice
1        Bob
2    Charlie
3      David
Name: Name, dtype: object
```

print(filtered_df)

[56]:
```
#Exercise 3: Filter Dataframe to include only rows where the age is greater than 25.
filtered_df = df[df['Age'] > 25]
print(filtered_df)
```

```
    Name  Age         City
1    Bob   27  Los Angeles
3  David   32      Houston
```

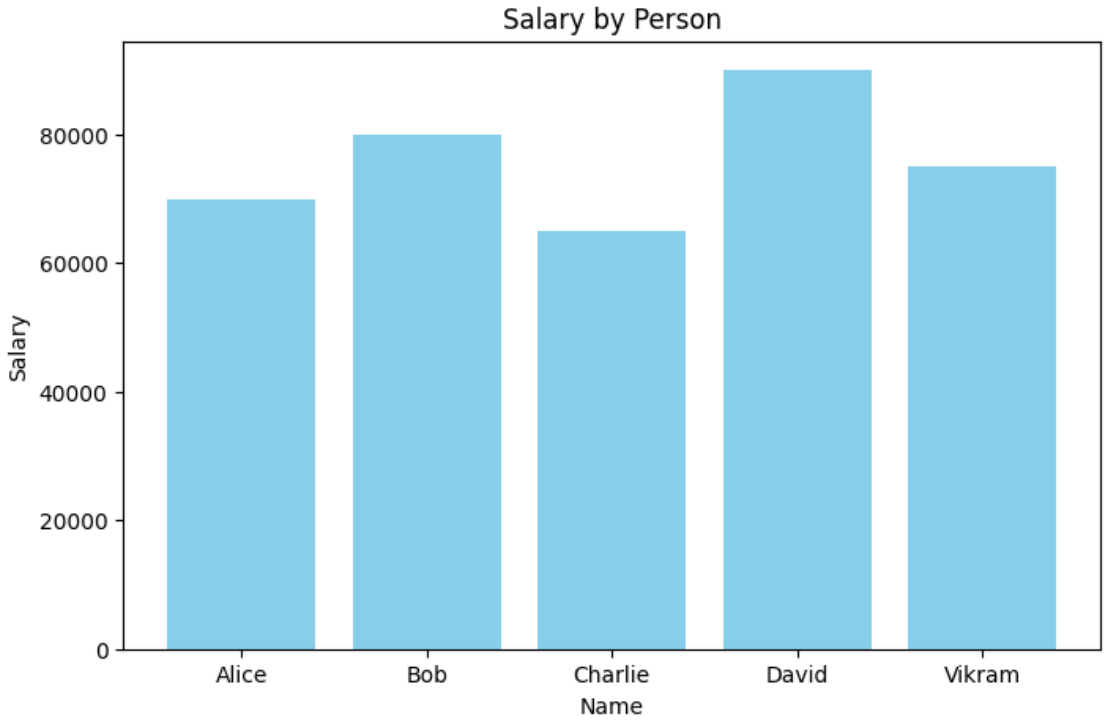
Practice:

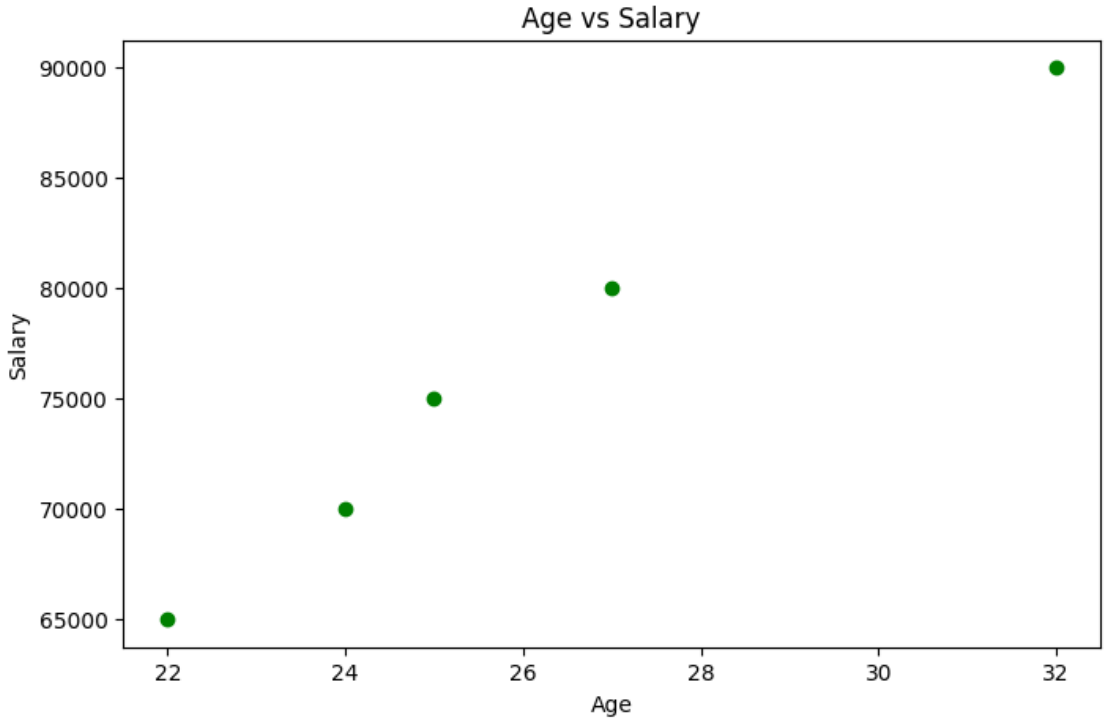
[57]:
```
#COMPLETE THIS CODE
#add your name and other information to the dataframe (name should be real)
import pandas as pd
import matplotlib.pyplot as plt

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [24, 27, 22, 32],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}
df = pd.DataFrame(data)

# Add my own row
df.loc[len(df)] = ['Vikram', 25, 'Edison']
print("DataFrame with my row added:")
print(df)

#Exercise 4: Add a New Column 'Salary' to the DataFrame with values [70000, 80000, 65000, 90000].
df['Salary'] = [70000, 80000, 65000, 90000, 75000]
print("\nWith Salary column:")
```

```
print(df)

#Exercise 5: Group by 'City' and calculate the average age.
avg_age_by_city = df.groupby('City')['Age'].mean()
print("\nAverage age by city:")
print(avg_age_by_city)

#Exercise 6: Visualize Data with Matplotlib
plt.figure(figsize=(8, 5))
plt.bar(df['Name'], df['Salary'], color='skyblue')
plt.xlabel('Name')
plt.ylabel('Salary')
plt.title('Salary by Person')
plt.show()

plt.figure(figsize=(8, 5))
plt.scatter(df['Age'], df['Salary'], color='green')
plt.xlabel('Age')
plt.ylabel('Salary')
plt.title('Age vs Salary')
plt.show()

#Exercise 7: Write this DataFrame into a CSV file and then Read it back from a␣
 ↪CSV File and store in another variable
df.to_csv('people.csv', index=False)
df_from_csv = pd.read_csv('people.csv')
print("\nDataFrame read back from CSV:")
print(df_from_csv)

#Exercise 8: Print first 5 rows and the summary.
print("\nFirst 5 rows:")
print(df.head())
print("\nSummary:")
print(df.describe())

#Exercise 9: Delete some data from the dataframe and then Handle that Missing␣
 ↪Data.
df_missing = df.copy()
df_missing.loc[1, 'Salary'] = None  # introduce a missing value
print("\nWith a missing value:")
print(df_missing)
# Handle missing data by filling with the mean salary
df_missing['Salary'] = df_missing['Salary'].fillna(df_missing['Salary'].mean())
print("\nAfter handling missing data (filled with mean):")
print(df_missing)
```

```python
#Exercise 10: Create a second dataframe and Merge two DataFrames on the 'Name' column.
df2 = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Vikram'],
    'Department': ['HR', 'Engineering', 'Marketing', 'Finance', 'IT']
})
merged_df = pd.merge(df, df2, on='Name')
print("\nMerged DataFrame:")
print(merged_df)
```

```
DataFrame with my row added:
      Name  Age         City
0    Alice   24     New York
1      Bob   27  Los Angeles
2  Charlie   22      Chicago
3    David   32      Houston
4   Vikram   25       Edison

With Salary column:
      Name  Age         City  Salary
0    Alice   24     New York   70000
1      Bob   27  Los Angeles   80000
2  Charlie   22      Chicago   65000
3    David   32      Houston   90000
4   Vikram   25       Edison   75000

Average age by city:
City
Chicago        22.0
Edison         25.0
Houston        32.0
Los Angeles    27.0
New York       24.0
Name: Age, dtype: float64
```


Salary by Person
80000
60000
40000
20000
0
Salary
Alice
Bob
Charlie
David
Vikram
Name


Age vs Salary
90000
85000
80000
75000
70000
65000
Salary
22
24
26
28
30
32
Age

```
DataFrame read back from CSV:
      Name  Age         City  Salary
0    Alice   24     New York   70000
1      Bob   27  Los Angeles   80000
2  Charlie   22      Chicago   65000
3    David   32      Houston   90000
4   Vikram   25       Edison   75000

First 5 rows:
      Name  Age         City  Salary
0    Alice   24     New York   70000
1      Bob   27  Los Angeles   80000
2  Charlie   22      Chicago   65000
3    David   32      Houston   90000
4   Vikram   25       Edison   75000

Summary:
              Age        Salary
count   5.000000      5.000000
mean   26.000000  76000.000000
std     3.807887   9617.692031
min    22.000000  65000.000000
25%    24.000000  70000.000000
50%    25.000000  75000.000000
75%    27.000000  80000.000000
max    32.000000  90000.000000

With a missing value:
      Name  Age         City   Salary
0    Alice   24     New York  70000.0
1      Bob   27  Los Angeles      NaN
2  Charlie   22      Chicago  65000.0
3    David   32      Houston  90000.0
4   Vikram   25       Edison  75000.0

After handling missing data (filled with mean):
      Name  Age         City   Salary
0    Alice   24     New York  70000.0
1      Bob   27  Los Angeles  75000.0
2  Charlie   22      Chicago  65000.0
3    David   32      Houston  90000.0
4   Vikram   25       Edison  75000.0

Merged DataFrame:
      Name  Age         City  Salary   Department
0    Alice   24     New York   70000           HR
1      Bob   27  Los Angeles   80000  Engineering
```

```
2  Charlie   22      Chicago   65000    Marketing
3    David   32      Houston   90000      Finance
4   Vikram   25       Edison   75000           IT
```